

Práctica 1: Representación de la información

1. Objetivos

- Conocer e identificar los formatos utilizados para representar datos de tipo numérico y caracteres en la computadora.
- Utilizar el lenguaje de programación C para verificar el uso de los formatos binario, complemento a 2, IEEE-754 y el código de representación ASCII.
- Verificar los rangos de los tipos de datos utilizados en el lenguaje de programación C.

2. Introducción

La **información** es el conjunto de datos organizados y procesados que constituyen mensajes, instrucciones, operaciones, funciones y cualquier tipo de actividad que tiene lugar en relación con la computadora. Podemos clasificar la información, primordialmente, en las siguientes categorías:

- **Datos numéricos:** generalmente combinaciones de dígitos del 0 al 9.
- **Datos alfabéticos:** compuestos solo por letras.
- **Datos alfanuméricos:** combinación de los dos anteriores.

Los datos numéricos se dividen en dos categorías: números enteros y números reales. Los números enteros, **con y sin signo**, se representan utilizando los siguientes formatos:

- Representación binaria
- Signo-magnitud
- Complemento a 2

Por otra parte, los números reales o de punto flotante utilizan la siguiente representación:

- IEEE-754

Finalmente, los datos **alfabéticos y alfanuméricos** se representan primordialmente utilizando el siguiente estándar de codificación:

- ASCII

2.1. Formato de representación binaria

En esta representación, un **número entero sin signo** se escribe como un **número binario** de n bits. Con n bits, el valor máximo que podemos representar es:

$$2^n - 1$$

Por lo tanto, en esta representación, un entero x de n bits puede tomar los valores:

$$0 \leq x \leq 2^n - 1$$

2.2. Formato de representación signo-magnitud

Sirve para representar **números enteros con signo** utilizando n bits, asignando el bit más significativo para almacenar el signo y el resto de los bits ($n - 1$) para almacenar la magnitud. El bit de signo se establece en 1 para un entero negativo y en 0 para un entero positivo. La magnitud se escribe como un número binario con $n - 1$ bits. En esta representación, un entero x de n bits puede tomar los valores:

$$-(2^{n-1} - 1) \leq x \leq +(2^{n-1} - 1)$$

En esta representación existen dos versiones del cero, es decir, ± 0 . Por ejemplo, para 8 bits:

$$00000000 = +0$$

$$10000000 = -0$$

2.3. Formato de representación complemento a 2

Sirve para representar **números enteros con signo**, particularmente los **números negativos**. En esta representación, el bit más significativo se utiliza para el signo y el resto de los bits ($n - 1$) para la magnitud.

Los números positivos se obtienen de la misma forma que en el formato signo-magnitud, mientras que los números negativos se obtienen invirtiendo los bits (ceros por unos y unos por ceros) y sumando una unidad al resultado.

El rango es **asimétrico**, por lo que se elimina el problema de tener dos representaciones para el cero. En esta representación, un entero x de n bits puede tomar los valores:

$$-2^{n-1} \leq x \leq +2^{n-1} - 1$$

2.4. Formato IEEE-754

Este estándar ofrece formatos que permiten representar números de punto flotante como vectores de bits. Consta de tres partes: **signo, exponente y mantisa**.

El formato de **precisión simple** utiliza 32 bits, donde el exponente se almacena en exceso 127, es decir, se le suma 127. Los campos del formato son los siguientes:

1 bit	8 bits	23 bits
signo	exponente	mantisa

Por otra parte, el formato de **doble precisión** utiliza 64 bits. En este caso, el exponente se almacena en exceso 1023, es decir, se le suma 1023. Los campos del formato son los siguientes:

1 bit	11 bits	52 bits
signo	exponente	mantisa

Para representar un número decimal fraccionario x en formato IEEE-754 se debe proceder de la siguiente manera:

1. Convertir x a binario.
2. Normalizar el resultado, dejando un 1 a la izquierda del punto.
3. Expresar x de la forma $(-1)^s m \cdot b^e$.
4. Calcular la mantisa tomando la parte fraccionaria de x y rellenando con ceros a la derecha hasta obtener el tamaño requerido.
5. Calcular el exponente.
6. Concatenar los campos resultantes y expresar el resultado en hexadecimal.

3. Desarrollo

1. Investigue el tamaño y el rango de representación de los siguientes tipos de datos en lenguaje C y complete la tabla 1.
2. Investigue el funcionamiento de los **campos de bits** y las **uniones** en lenguaje C.
3. En algún IDE para lenguaje C/C++ de su preferencia (Visual Studio Code, CodeBlocks, Dev-C++, etc.), cree un nuevo archivo titulado `bytechar.cpp` y escriba la estructura básica de un programa en C.

Tipo de dato	Tamaño (bytes)	Tamaño (bits)	Rango de representación
char			
unsigned char			
short			
unsigned short			
int			
unsigned int			
long			
float			
double			

Tabla 1: Tamaño de algunos tipos de datos en lenguaje C.

4. Para poder observar el contenido de un byte en memoria, es necesario utilizar campos de bits. Para ello, cree una estructura que contenga ocho miembros del tipo **unsigned char**. Nombre los miembros iniciando en **bit7** y terminando en **bit0**, indicando que en cada campo solo se utilizará un bit.

```
struct byte {
    unsigned char bit7:1;
    unsigned char bit6:1;
    unsigned char bit5:1;
    unsigned char bit4:1;
    unsigned char bit3:1;
    unsigned char bit2:1;
    unsigned char bit1:1;
    unsigned char bit0:1;
};
```

5. Con ayuda de **typedef**, cree un alias para la estructura de campos de bits, renombrándola como **Byte**.

```
typedef struct byte Byte;
```

6. Cree una unión con dos miembros: una variable de tipo **char** y otra del tipo **Byte**.

```
union bytechar {
    char c;
    Byte b;
};
```

7. Al igual que con la estructura de campos de bits, utilice **typedef** para renombrar la unión como un nuevo tipo de dato llamado **ByteChar**.

```
typedef union bytechar ByteChar;
```

8. Cree una función de apoyo que permita imprimir en la salida estándar el contenido de un dato de tipo **Byte**.

```
void printByte(Byte b) {
    printf("%d",b.bit0);
    printf("%d",b.bit1);
    printf("%d",b.bit2);
    printf("%d",b.bit3);
    printf("%d",b.bit4);
    printf("%d",b.bit5);
    printf("%d",b.bit6);
    printf("%d",b.bit7);
}
```

9. En la función principal, declare una variable de tipo **ByteChar** y solicite al usuario un carácter para almacenarlo en el miembro de tipo **char** de la unión. Posteriormente, utilice la función **printByte** para mostrar en pantalla el contenido del miembro de tipo **Byte**. Con ello podrá observar los bits que componen el carácter almacenado en la unión.

```

int main() {
    ByteChar a;
    printf("Ingrese un caracter: ");
    scanf("%c",&a.c);
    printf("Character: %c\nHex: %X\nBinary: ",a.c,a.c);
    printByte(a.b);
    printf("\nSize: %d bytes\n\n",sizeof(char));
    return 0;
}

```

10. Investigue los valores ASCII de los caracteres presentes en la tabla 2 y complétela representando los valores en los formatos solicitados. Utilice el programa anterior para llenar la cuarta columna y verificar sus resultados.

Caracter	Valor ASCII Hex	Valor ASCII Binario	Resultado del programa	¿Coincide?
a				
s				
w				
C				
X				
5				
8				
?				
+				
{				

Tabla 2: Caracteres ASCII y su representación.

11. Convierta los números de la tabla 3 a binario, representándolos con 32 bits, y complete la tabla con los formatos indicados. Cree un nuevo programa titulado **byteint.cpp** que muestre en pantalla el contenido de los 4 bytes que componen una variable de tipo **int** en lenguaje C, utilizando la estructura **Byte**.

Decimal N	N Binario	+N Signo-magnitud	-N Complemento a 2	Resultado del programa
5				
10				
96				
128				
522				
1023				
4001				
15000				
115200				
10524758				

Tabla 3: Números enteros y su representación.

12. Cree un nuevo programa titulado **bytefloat.cpp** que muestre en pantalla el contenido de los 4 bytes que componen una variable de tipo **float**. Con ayuda del programa, llene la tabla 4 indicando el signo, el exponente y la mantisa.
13. Cree un nuevo programa titulado **bytedouble.cpp** y repita el punto anterior utilizando una variable de tipo **double** y los datos de la tabla 4.
14. Anote y reporte sus observaciones. Debe entregar sus códigos de comprobación: **bytechar.cpp**, **byteint.cpp**, **bytefloat.cpp** y **bytedouble.cpp**.

Número real	IEEE-754 Precisión simple Hexadecimal	Signo (s) 1 bit	Exponente (e) 8 bits	Mantisa (m) 23 bits
+1.75				
-0.5				
-12.09375				
+120.125				
-0				

Tabla 4: Números de punto flotante y su representación.

Bibliografía recomendada

- [1] M. Morris Mano y Michael D Ciletti. *Diseño Digital*. 5th edition. Pearson Education, 2013. ISBN: 9786073220408.
- [2] William Stallings. *Organizacion Y Arquitectura De Computadores*. 7th edition. Prentice Hall Press, 2006. ISBN: 9788489660823.
- [3] Andrew S. Tanenbaum y Todd Austin. *Structured Computer Organization*. 6th edition. USA: Prentice Hall Press, 2012. ISBN: 0132916525.